

Debugging Techniques for Performance Optimisation

Learning Outcomes

- Define a performance symptom precisely, and reproduce it reliably before touching any code.
- Use `breakpoint()` and `pdb` to pause a running program and inspect its actual state.
- Use conditional breakpoints to isolate rare or data-dependent behaviour, without stopping thousands of times.
- Add targeted timing, counters, and structured logging that answer one question without overwhelming the workload.
- Use `tracebacks` and `faulthandler` to investigate difficult runtime problems, including hangs and crashes.
- Diagnose excessive calls, repeated work, retries, and inefficient loops using simple instrumentation.
- Investigate memory growth using `tracemalloc`, and distinguish expected allocation from accidental retention.
- Fix the root cause, verify correctness, and measure the improvement, rather than declaring victory on instinct.

Performance Debugging vs Profiling

The two activities ask different questions, and real investigations move between both.

Profiling asks

- Where is CPU time being spent?
- Where is memory being allocated?
- Which function or line dominates the total cost?

Debugging asks

- Why is the program behaving this way?
- What input or state actually triggers the cost?
- Is a specific hypothesis about the cause correct?

A Deliberately Slow Workload

Save as `slow_app.py` and run: `python slow_app.py`

- Record the baseline before changing anything - this is the number every later fix gets compared against.
- The code is correct; the investigation is about cost, not about finding a bug in the output.
- Do not assume the string conversion is the bottleneck - measure it before believing it.

```
slow_app.py
import time

def process(values):
    total = 0
    for value in values:
        # Repeated conversion inside a hot loop
        text = str(value)
        total += int(text)
    return total

if __name__ == "__main__":
    values = range(500_000)
    start = time.perf_counter()
    result = process(values)
    elapsed = time.perf_counter() - start

    print("result:", result)
    print("elapsed:", elapsed)
```

CONCEPTS

Using breakpoint() and pdb

breakpoint() pauses execution and drops into pdb, Python's built-in interactive debugger.

- Why use it: a profiler tells you a function is slow, but only a debugger lets you stop mid-execution and ask what the actual values are at that exact moment.
- What the session shows you: the current line, the local variables reachable from there, and the full call stack that led to this point.
- Interactive debugging is for understanding state - it is not a precise timing tool, since stopping execution changes its timing completely.
- Once the state makes sense, use `c` to continue, or `q` to quit the session entirely.



`pdb` commands

```
p expression      # print the value of an expression
pp expression     # pretty-print a complex object
n                 # execute the next source line
s                 # step INTO a function call
w (or where)      # show the current call stack
c                 # continue running to the next breakpoint
q                 # quit the debugger entirely
```

Hands-On: Inspect Suspicious State

Run the example. Execution will pause automatically the moment a negative value is encountered.

- At the breakpoint, try: p index, then p value and p total, to see exactly what triggered the pause.
- Use where to confirm which function called this one, and from where.
- Use n to advance one line at a time and watch total change.
- Use c to let the loop continue once you understand what happened.
- What this tells you: the debugger shows the real values at the real moment of interest, instead of you guessing from the code alone.

```
inspect_suspicious_state.py
def calculate(values):
    total = 0
    for index, value in enumerate(values):
        if value < 0:
            breakpoint()
            total += value * value
    return total

values = [2, 4, 7, -3, 8]

print("result:", calculate(values))
```

Conditional Debugging in a Large Loop

Stop only when the state actually matches your investigation, not on every iteration.

- Why use it: without a condition, a loop of 100,000 iterations would mean pressing continue 100,000 times - a plain breakpoint() is unusable here.
- A condition lets the loop run at full speed until the exact case worth inspecting occurs.
- Use this for rare or data-dependent bugs, where the interesting case is buried inside routine ones.
- Do not use a debugger to make fine-grained performance claims - pausing execution changes its timing, so what you see is no longer representative.

```
conditional_breakpoint.py
def process(values):
    total = 0
    for index, value in enumerate(values):
        if index == 10_000 and value > 90_000:
            breakpoint()
        total += value
    return total

values = list(range(100_000))
values[10_000] = 95_000 # ensures the condition can fire
print(process(values))
```

Call-Count Instrumentation

Determine whether a suspicious function is simply being called far more often than expected.

- Why use it: a counter answers the question “how often?” - something neither a debugger nor a stopwatch tells you directly.
- What the result means: a high call count on its own is not proof of a problem, but it tells you where to look next.
- High call frequency can dominate total cost even when each individual call is cheap.
- Combine call counts with cProfile or direct timing to convert “called a lot” into “costs this many seconds.”
- The next question to ask is always: are all of these calls actually necessary?

```
call_count_instrumentation.py

calls = 0

def normalise(value):
    global calls
    calls += 1
    return str(value).strip().lower()

def process(values):
    return [normalise(value) for value in values]

values = range(100_000)
result = process(values)
print("items:", len(result))
print("normalise calls:", calls)
```


Targeted Timing with perf_counter()

Measure only the operation relevant to your current hypothesis.

- Why use it: perf_counter() gives a direct, simple elapsed-time answer for exactly the code wrapped inside it, with no report to interpret.
- What the result means: total_time is the sum of time spent specifically inside normalise, isolated from everything else the program does.
- Use timeit instead when comparing small alternatives head-to-head, rather than measuring one real function in place.
- If the difference between two numbers looks small, repeat the measurement before trusting it.

```
targeted_timing.py
import time

def normalise(value):
    start = time.perf_counter()
    result = str(value).strip().lower()
    elapsed = time.perf_counter() - start
    return result, elapsed

total_time = 0.0
for value in range(50_000):
    _, elapsed = normalise(value)
    total_time += elapsed

print("measured normalise time:", total_time)
```

Structured Logging for Performance Diagnosis

Run: `python logging_demo.py`

- Why use it: unlike a one-off `print()`, logging gives repeatable diagnostic evidence that survives in production and can be searched later.
- What the result means: each log line records exactly how many items were processed and how long it took, so trends across runs become visible.
- Include workload size alongside elapsed time - 0.6 seconds means something different for 100 items than for 1,000,000.
- Avoid logging every iteration of a hot loop - that logging call itself can become the new bottleneck.
- Use `DEBUG` level for detailed diagnostics that stay off by default, and `INFO` for evidence worth keeping routinely.

```
logging_demo.py
import logging
import time

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s %(message)s"
)
logger = logging.getLogger(__name__)

def process(values):
    start = time.perf_counter()
    result = [value * 2 for value in values]
    elapsed = time.perf_counter() - start
    logger.info(
        "process items=%d elapsed=%.6fs",
        len(values), elapsed,
    )
    return result

process(range(100_000))
```

Sampling Logs in a Hot Loop

Reduce diagnostic overhead while still retaining visibility into progress.

- Why use it: logging on every one of 100,000 iterations would slow the loop down and flood the log file with noise.
- What the result means: the modulo check means only 10 progress messages are ever emitted, evenly spaced across the whole run.
- Sampling is often a better default than logging every iteration - you still see whether progress is steady or stalling.
- Diagnostic code should never become the performance problem it was added to investigate.
- Keep sensitive values out of logs, even at DEBUG level - a diagnostic log can end up somewhere less controlled than expected.

```
sampling_logs.py
import logging

logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

for index, value in enumerate(range(100_000)):
    if index % 10_000 == 0:
        logger.debug(
            "progress index=%d value=%d",
            index, value,
        )
    # application work
```

Exceptions as a Performance Clue

Measure whether an exception-heavy code path is occurring more often than expected.

- Why use it: counting attempts against failures turns a vague suspicion (“this feels slow”) into a concrete failure rate.
- What the result means: a failure rate of 0.4 says two in every five inputs are hitting the exception path - that is worth investigating on its own.
- Exceptions carry a real runtime cost, which matters especially when they occur frequently rather than exceptionally.
- Prefer exceptions for genuinely exceptional conditions rather than routine control flow, where that is practical.
- Measure the actual failure rate before redesigning any error handling around it.

```
exceptions_performance_clue.py

attempts = 0
failures = 0

def parse_value(text):
    global attempts, failures
    attempts += 1
    try:
        return int(text)
    except ValueError:
        failures += 1
    return 0

values = ["10", "20", "bad", "30", "bad"]
result = [parse_value(value) for value in values]
print("attempts:", attempts)
print("failures:", failures)
print("failure rate:", failures / attempts)
```

Traceback Diagnostics

Use traceback information to identify exactly where repeated failures originate.

- Why use it: a traceback names the precise file, line, and call chain that produced the error, rather than leaving you to guess.
- What the result means: reading top to bottom shows the path execution took to reach the failure, one call at a time.
- For production applications, structured logging of the traceback is usually preferable to printing it directly.
- Use the traceback to understand the execution path that led to the problem, not only the final error message.
- Do not simply suppress repeated exceptions without first understanding why they are happening.

```
traceback_diagnostics.py
import traceback

def divide(values):
    result = []
    for value in values:
        try:
            result.append(100 / value)
        except ZeroDivisionError:
            traceback.print_exc()
            result.append(None)
    return result

print(divide([10, 5, 0, 20]))
```

faulthandler for Difficult Runtime Problems

Run: python faulthandler_demo.py

- Why use it: some crashes and hangs happen below the level normal Python exceptions can report on - faulthandler is built specifically for these.
- What it gives you: a traceback for certain low-level crashes, and the ability to dump every thread's current stack on demand.
- It is included with Python, so there is nothing extra to install.
- It complements normal debugging and profiling rather than replacing either of them.
- Enable it deliberately in diagnostic environments, when a problem is otherwise hard to reproduce with a debugger alone.

```
faulthandler_demo.py
import faulthandler
import time

faulthandler.enable()

print("Program started")
# Simulate a long-running operation
time.sleep(2)
print("Program finished")
```

Memory Debugging with tracemalloc

Compare memory snapshots taken before and after the workload runs.

- Why use it: CPU timing tools say nothing about memory - tracemalloc answers the separate question of where allocation is actually happening.
- What the result means: each printed stat shows a source line and how much its allocations changed between the two snapshots.
- `compare_to()` highlights the differences between snapshots, which is far more informative than either snapshot alone.
- 'lineno' groups those differences by the exact source line responsible for the allocation.
- Ask, for each large entry: should these objects still be alive at this point, or should they have already been released?

```
tracemalloc_memory_debugging.py
import tracemalloc

tracemalloc.start()
before = tracemalloc.take_snapshot()

data = [
    {"id": i, "payload": str(i) * 10}
    for i in range(50_000)
]

after = tracemalloc.take_snapshot()
for stat in after.compare_to(before, "lineno")[:10]:
    print(stat)

print("objects retained:", len(data))
tracemalloc.stop()
```

Diagnosing Accidental Retention

A cache or global collection can keep objects alive for the entire life of the process.

- Why this matters: the dictionary is still reachable from a live variable, so nothing about it looks wrong to Python - every entry stays alive on purpose, by design.
- What to check: growing memory usage is not automatically a leak - the real question is whether every one of those 100,000 entries is still needed.
- Investigate whether the cache needs bounds, an eviction policy, or a maximum size.
- Use tracemalloc (previous slide) to confirm where that growing memory is actually going, rather than assuming.

```
diagnosing_accidental_retention.py
cache = {}

def add_result(key, value):
    cache[key] = value

for i in range(100_000):
    add_result(i, {"value": i})

print("cache size:", len(cache))

# Diagnostic question:
# Does the application really need every entry?
```


Diagnosing Inefficient Membership Tests

Instrument the workload first, before deciding to change its data structure.

- Why instrument first: checks and matches turn “this loop feels slow” into an exact count of how much membership-testing work is actually happening.
- What the result means: one membership check happens per input value, so the total work scales directly with how large values is.
- List membership (value in targets) is generally O(n) - each check may have to scan much of the list.
- A large targets collection makes that per-check cost, multiplied by every value, expensive in total.
- Confirm with profiling or timing that this really is a meaningful hotspot before optimising it.

```
diagnosing_membership_tests.py

def count_matches(values, targets):
    checks = 0
    matches = 0
    for value in values:
        checks += 1
        if value in targets:
            matches += 1
    return matches, checks

values = range(100_000)
targets = list(range(0, 100_000, 10))
matches, checks = count_matches(values, targets)
print("matches:", matches)
print("membership checks:", checks)
```

From List to Set Membership

Measure both implementations directly, and confirm the fix preserves correctness.

- Why measure both: a claimed speedup means nothing without a baseline measured under the same conditions.
- What the result means: $\text{speedup} = \text{slow_time} / \text{fast_time}$ - a result of 5.0 means the optimised version took one-fifth the time.
- Set membership (`value in target_set`) is generally $O(1)$ on average, unlike list membership's $O(n)$.
- Converting targets to a set has its own cost, so that conversion is included inside `fast()`, not hidden outside the measurement.
- The `assert` statement is not optional - it proves the two versions still agree before any speed claim is trusted.

```
list_to_set_fix.py

import time

def slow(values, targets):
    return sum(value in targets for value in values)

def fast(values, targets):
    target_set = set(targets)
    return sum(value in target_set for value in values)

values = range(200_000)
targets = list(range(0, 200_000, 10))

start = time.perf_counter()
a = slow(values, targets)
slow_time = time.perf_counter() - start

start = time.perf_counter()
b = fast(values, targets)
fast_time = time.perf_counter() - start

assert a == b

print("slow:", slow_time)
print("fast:", fast_time)
print("speedup:", slow_time / fast_time)
```

Debugging I/O-Like Latency

CPU debugging alone may not explain user-visible waiting time.

- Why this matters: everything covered so far measures Python CPU work - this measures waiting, which is a completely different kind of cost.
- What the result means: elapsed here is dominated by `time.sleep()`, not by any Python computation - a CPU profiler would show almost nothing happening.
- Measure external boundaries explicitly, at the point where your code calls out and waits for a response.
- In real systems, combine Python-level diagnostics with database, network, and external-service metrics.

```
io_latency_debugging.py
import time

def external_operation():
    # Simulate an external wait
    time.sleep(0.05)
    return "ok"

start = time.perf_counter()
result = external_operation()
elapsed = time.perf_counter() - start
print("result:", result)
print("external wait:", elapsed)
```

Copy-and-Paste Template

Save as `performance_debug.py` and run: `python performance_debug.py`

- Replace `workload()` with the operation actually being investigated - everything around it stays the same.
- Use `breakpoint()` inside `workload()` whenever state inspection, not just timing, is needed.
- Use `cProfile` separately when a full CPU call hierarchy is needed - this template favours a quick, direct check.
- Keep diagnostics reproducible and easy to remove once the investigation is finished.

```
performance_debug.py

import logging
import time
import tracemalloc

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def workload():
    # Replace this with the operation under investigation.
    return sum(i * i for i in range(100_000))

if __name__ == "__main__":
    tracemalloc.start()
    before = tracemalloc.take_snapshot()

    start = time.perf_counter()
    result = workload()
    elapsed = time.perf_counter() - start

    after = tracemalloc.take_snapshot()

    logger.info("elapsed=%.6fs result=%s", elapsed, result)

    for stat in after.compare_to(before, "lineno")[:5]:
        logger.info("memory: %s", stat)

    tracemalloc.stop()
```